

Media Recommendation Assistant

A chatbot-style workflow that turns a plain-English client brief into a reviewable Kargo product or bundle recommendation.

Media strategists need fast, defensible recommendations.

Input is messy

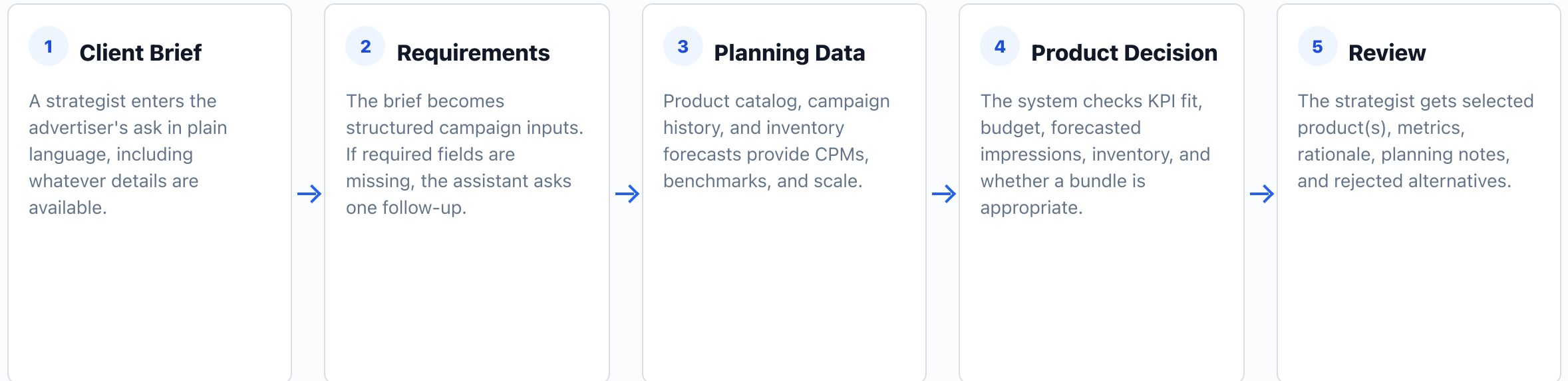
Client briefs arrive as natural language. Some are complete, while others omit KPI, budget, geo, vertical, or impression goals.

Recommendation must be auditable

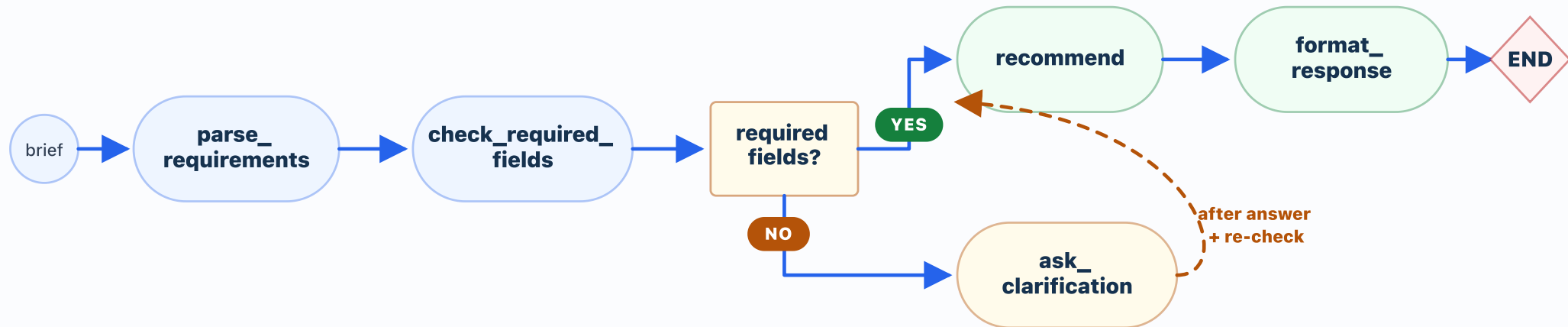
The final answer has to explain the selected product or bundle, calculated KPI benchmark, forecasted impressions, inventory fit, and rejected alternatives.

The assistant should help a strategist review the answer quickly, not hide the math behind a black box.

How the assistant turns a brief into a recommendation.



LangGraph routes complete and incomplete briefs differently.



The assistant identifies or asks for the fields needed to recommend.

Required

- Vertical: Retail, Finance, Travel, QSR, Entertainment
- Primary KPI: CTR or in-view rate
- Geo: US, EMEA, APAC
- Budget in dollars

Optional

- Advertiser name, when available
- Impression goal
- Single-product or bundle intent

Clarification behavior

When a required field is missing, the graph stops before recommendation and asks a targeted follow-up question.

Each file has a narrow job in the recommendation.

RESOURCE	USED FOR	DECISION IMPACT
client_briefs.json	Sample plain-text user inputs.	Exercises complete briefs, missing-field clarification, and tradeoff scenarios.
product_catalog.csv	Product names, descriptions, and CPM.	CPM drives forecasted impressions for the requested budget.
campaign_history.csv	Historical impressions, clicks, and viewable impressions by product and vertical.	Calculates product benchmarks for CTR and in-view rate.
inventory_forecaster.csv	Available impressions by product, vertical, geo, and inventory confidence.	Determines whether a product has enough usable scale for budget and goals.

The decision layer is deterministic and testable.

```
CTR = clicks / impressions
```

```
In-view rate = viewable_impressions / impressions
```

```
Estimated impressions = budget / CPM * 1000
```

```
Usable inventory = availableimps * inventory_confidence
```

Ranking

Products are ranked by the client's primary KPI, with deterministic tie-breakers for inventory confidence, scale, CPM, and product ID.

Viability

A product can be rejected or caveated when it cannot absorb the budget, misses an impression goal, or lacks usable inventory.

Inventory risk is handled as confidence, not ignored.

Implementation choice

The file describes `inventory_risk` as a forecast confidence score from 0.0 to 1.0, where higher means more confidence. The engine treats it as inventory confidence.

```
usable_inventory = available_imps *  
inventory_confidence
```

Why this is useful

It makes scale checks conservative. A product with large available impressions but low confidence will not look as strong as the raw forecast suggests.

The engine optimizes for the brief's intent.

Bundle allowed

Default behavior. Recommend a small bundle when it improves KPI performance and budget delivery.

Single product preferred

Used when the brief asks for a simple, straightforward, one-product, or clearly single recommendation.

Maximize delivery

Used when the brief explicitly asks to use the full budget, maximize scale, or spend as much as possible.

The LLM infers intent, but the deterministic engine still decides which product or bundle wins.

The final answer is structured for quick inspection.

Stellar Bank is a Finance advertiser running in the US with a \$20,000 budget. They care most about click-through rate and want at least 800,000 impressions.

Recommend Premium Takeover + Commerce Connect. This bundle has the strongest blended click-through rate at 1.2% and uses the full \$20,000 budget.

1.2%
BLENDED CTR

800K
FORECASTED IMPRESSIONS

\$20K
BUDGET USED

2
PRODUCTS SELECTED

Why this is a bundle

The brief does not ask for one product, so the engine combines Premium Takeover and Commerce Connect to hit the impression goal while keeping the blended CTR strong.

Top rejected alternative

Attention Builder forecasts more impressions as a single product, but its CTR is lower than the selected bundle: 0.8% vs. 1.2%.

When the brief asks for one clear product, the engine stays focused.

Acme Shoes wants one straightforward Retail recommendation in the US with a \$30,000 budget focused on click-through rate.

Recommend Premium Takeover. It has the strongest viable CTR benchmark at 0.9% and can use the full \$30,000 budget.

0.9%

CTR BENCHMARK

938K

FORECASTED IMPRESSIONS

\$30K

BUDGET USED

941K

USABLE INVENTORY

Why this stays single-product

The brief says "one straightforward recommendation," so the LLM parser sets single-product intent and the deterministic engine chooses the best viable product.

Top rejected alternative

Commerce Connect has higher CTR at 1.2%, but it cannot absorb the full \$30,000 as a single product: 1.4M usable impressions vs. 2M needed.

Use the LLM where ambiguity lives. Use code where correctness matters.

LLM responsibilities

- Parse messy brief text into structured fields.
- Merge follow-up answers with existing requirements.
- Infer planning intent: bundle allowed, single product, or maximize delivery.

Code responsibilities

- Benchmark calculations and CPM math.
- Inventory confidence and scale checks.
- Ranking, bundle allocation, rejection reasons, and tests.

Pros and cons of agentic versus mostly programmatic.

Pros of agentic / LLM-assisted

- Handles natural-language briefs and incomplete phrasing.
- Can infer soft intent like "straightforward recommendation."
- Produces a natural clarification loop without brittle regex rules.
- Can be extended to tool routing if the workflow grows.

Cons if fully agentic

- Harder to guarantee reproducible math and ranking.
- More risk of hallucinated product reasoning.
- Harder to unit test every decision path.
- Potentially slower and more expensive for simple calculations.

The final architecture is intentionally hybrid: agentic workflow for orchestration and parsing, deterministic Python for media planning decisions.

Confidence comes from deterministic tests plus opt-in live checks.

65

TESTS PASSING

4

LIVE TESTS SKIPPED BY DEFAULT

0

DEFAULT OPENAI CALLS IN TESTS

Many

SYNTHETIC BRIEF SCENARIOS

Covered paths

Data loading, benchmark math, recommendation status, bundle logic, stale capacity reason normalization, UI helper formatting, parser behavior, and workflow routing.

Live LLM tests

OpenAI calls are opt-in via `RUN_LIVE_OPENAI_TESTS=1`, which keeps normal tests fast and deterministic.

The implementation is practical, but intentionally scoped.

No time dimension

Inventory has no dates or pacing. The assistant treats inventory as a static forecast because the case data does.

Simple benchmarks

Benchmarks are historical aggregate rates by product and vertical. There is no seasonality or learned model.

Small catalog

The planner is optimized for the given product catalog and case-study data, not a full production inventory graph.

This is a case-study assistant, not an automated media buyer. It is built to make the recommendation easy to inspect.

What I would add next for production readiness.

Planning quality

- Add date-aware inventory and pacing.
- Allow configurable scoring weights by client goal.
- Support product exclusions, must-have products, and frequency constraints.
- Use strategist feedback to calibrate recommendations.

Engineering quality

- Add a lightweight eval dashboard for parser failures.
- Persist recommendation traces for review.
- Add deployment configuration and secret management.
- Export recommendation summaries to PDF or slides.

Discussion prompts

Product and strategy

- When should a bundle beat a single product?
- How much should inventory confidence penalize scale?
- What should a strategist be able to override?

Technical depth

- Which parts should remain deterministic?
- How would we evaluate parser quality?
- What changes for production deployment?